SCALER | Priyansh Saxena

LLM Serving and Inference Optimization

Memory Bounds ▪ KV Cache ▪ Quantization ▪ Batching ▪ Deployment

Priyansh Saxena

Scaler School of Technology | ML System Design and LLMOps

Week 3 — Session 1/2

ROADMAP

Agenda

1. Why LLM Inference Is Hard
2. KV Cache and Memory Management
3. Quantization
4. Batching and Speculative Decoding
5. Deployment Choices

TOC

1

Why LLM Inference Is Hard

1 The memory problem: GPT-3 by the numbers

Just to load the model weights

- GPT-3 has **175 billion** parameters
- At fp16 (2 bytes/param): 175B $\times$ 2 = **350** GB RAM
- A top-of-the-line A100 GPU has **80 GB** VRAM
- Therefore: minimum **5**$\times$ **A100s** just for weights
- Add KV cache, activations, framework overhead...

Contrast with training

- During training you can accumulate gradients across batches
- At inference you must hold the *entire* model in-flight for every request
- No gradient checkpointing trick helps here

Quick sanity check

Model	Params	fp16 RAM
GPT-2	1.5B	3 GB
LLaMA-2-7B	7B	14 GB
LLaMA-2-13B	13B	26 GB
LLaMA-2-70B	70B	140 GB
GPT-3	175B	350 GB

Formula: params $\times$ bytes/param = RAM
fp32 = 4 B/param, fp16 = 2 B/param

1 Sequential generation: why you can't just parallelize

How autoregressive generation works

- Token t_1 is sampled from the model
- Token t_2 depends on t_1 — must wait
- Token t_k depends on $t_1, \ldots, t_{k-1}$
- This is **strictly sequential** — unlike training batches

What this means for hardware

- GPU is great at doing *many things at once*
- But each new token requires a full forward pass
- Even on an A100: ∼20–50 tokens/second for a 7B model
- 500-token response ≈ 10–25 seconds single-threaded

Training vs. Inference parallelism

Training: process a whole batch of 256 sequences in parallel. All sequences are known in advance. Fully parallelisable.

Inference: each token depends on the previous one. You *cannot* know token t_k before computing t_{k-1}. Sequential dependency is baked into the autoregressive formulation.

The only escape

Run *multiple requests* in parallel (batching). That is where GPU utilisation comes from — not from one long sequence.

1 Memory-bandwidth bound, not compute-bound

Roofline model for LLM inference

- **Compute-bound**: the bottleneck is floating-point operations (FLOPs). More compute ⇒ faster.
- **Memory-bandwidth bound**: the bottleneck is how fast you can move weights from HBM memory to compute units.
- For large LLMs: generating a single token requires **reading all weights once**
- A100 memory bandwidth: 2 TB/s. At 350 GB weights, that's ∼175 ms just to load weights per token!

Why this matters for optimization

- Reducing FLOPs (pruning, distillation) helps less than expected
- Reducing **model size in memory** (quantization) helps a lot
- Improving **memory access patterns** (fused kernels, FlashAttention) helps a lot
- Simply buying a "faster GPU" helps less than buying one with higher **memory bandwidth**

Key insight: the path to faster LLM inference goes through memory, not FLOPs.

1 The cost math: tokens × price × volume

Back-of-envelope: GPT-3.5 at scale

- GPT-3.5 Turbo: $0.002 per 1K tokens (2023 pricing)
- Average response: 1,000 tokens
- 1 million queries per day
- Cost: $1M \times 1K \times \$0.002/1K = \mathbf{\$2{,}000}$/day
- Monthly: **$60,000**
- Annual: **$730,000**

Modern pricing (2024)

- GPT-4o: $15/1M input tokens + $60/1M output tokens
- At 10M queries/day, 1K tokens each: **$600K/day**
- This is why LLM inference optimization is a $1B engineering problem

The 3 levers to pull costs down

1. **Fewer tokens** — prompt compression, caching, fewer retries
2. **Cheaper model** — right-size the model to the task; a 7B fine-tune often beats GPT-4 at a fraction of the cost
3. **Self-host** — amortise GPU cost over volume; break-even is roughly $10K/month API spend

2

KV Cache and Memory Management

2 What is the KV Cache and why do we need it?

Transformer attention — the expensive part

- Every attention layer computes **Keys** (K) and **Values** (V) for every token
- Attention: $\text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$
- At token t, you need K and V for **all** tokens $1 \ldots t-1$

Without KV cache

- Generating token N requires re-computing K, V for all $N-1$ previous tokens
- Cost per token grows **linearly** with sequence length
- 1000-token response: the last token costs 1000× more to compute than the first

With KV cache

- Compute K, V for token t once, store them
- When generating token $t+1$, retrieve stored K, V tensors
- Only compute K, V for the *new* token
- Cost per token: **constant** regardless of sequence length
- Standard in every production LLM serving system

Trade-off: saves compute, costs memory.

2 KV cache memory: the numbers are alarming

KV cache size formula

$$\text{KV mem} = 2 \times L \times H \times d \times S \times \text{bytes}$$

- L = number of layers, H = attention heads
- d = head dimension, S = sequence length
- Factor 2: one K tensor, one V tensor

For GPT-3 (fp16)

- $L = 96$ layers, $H = 96$ heads, $d = 128$
- $S = 2048$ context, 2 bytes/value
- KV cache $\approx$ **100** GB **per request**
- With 10 concurrent requests: **1 TB** of KV cache alone

The real problem

You can't pre-allocate KV cache for all possible requests. If you pre-allocate max-length buffers, most memory is **wasted** — a 100-token request wastes 95% of its 2048-token allocation. This is **memory fragmentation**: the leading cause of low GPU utilisation in naive LLM serving.

Practical numbers (7B model, fp16)

Weights	14 GB
KV cache per req (2K ctx)	$\sim$2 GB
10 concurrent requests	$\sim$20 GB

2 PagedAttention: OS paging for KV cache

The insight from operating systems

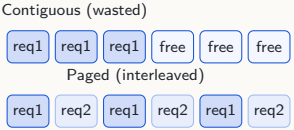
- OS virtual memory: physical RAM doesn't need to be contiguous
- Allocate in fixed-size **pages**; map pages to virtual addresses
- Pages can be anywhere in physical memory; no fragmentation

PagedAttention (vLLM, 2023)

- KV cache divided into fixed-size **blocks** ("pages")
- Each block holds K, V for a fixed number of tokens
- Blocks are allocated *on demand* as generation proceeds
- Blocks can be shared across parallel beams (beam search)
- Result: near-zero internal fragmentation

What PagedAttention achieves

- Memory fragmentation reduced by up to **90%**
- Enables **24× higher throughput** vs. HuggingFace Transformers on same hardware
- Supports much larger batch sizes
- Copy-on-write for shared prefixes (system prompts)



2 vLLM: putting it together

vLLM is a high-throughput LLM serving library

- Built at UC Berkeley (Kwon et al., 2023)
- Core innovation: **PagedAttention**
- Continuous batching built-in (more on this soon)
- Supports most major open-weight models (LLaMA, Mistral, Qwen, ...)
- OpenAI-compatible API server out of the box

Throughput comparison (same A100)

- HuggingFace Transformers: ∼5 req/s
- vLLM with PagedAttention: ∼120 req/s
- **24× improvement** with the same hardware budget

What makes vLLM fast

Feature	Benefit
PagedAttention	No KV waste
Continuous batching	Higher GPU util
CUDA kernels	Fused ops
Tensor parallel	Multi-GPU support
Prefix caching	Shared system prompts

Production reality

vLLM (or its equivalent: TGI, SGLang) is the default choice for self-hosted open-weight LLM serving. If you are evaluating self-hosting, start here.

3

Quantization

3 Precision levels: fp32 to INT4

Floating-point precision options

- **fp32** (full): 4 bytes/param. Gold standard. Used in training.
- **fp16** (half): 2 bytes/param. 2× smaller. Minor accuracy loss. Standard for inference.
- **bf16**: 2 bytes/param, wider exponent range than fp16. Better for training stability.
- **INT8**: 1 byte/param. 4× smaller than fp32. Works well for most tasks.
- **INT4**: 0.5 bytes/param. 8× smaller. Surprisingly good quality.
- **INT2**: 0.25 bytes/param. Usually degrades too much.

Memory at 70B params

Precision	Bytes	70B RAM
fp32	4.0	280 GB
fp16	2.0	140 GB
INT8	1.0	70 GB
INT4	0.5	35 GB
INT2	0.25	18 GB

INT4: a 70B model fits on a **single 40 GB A100**. That is transformative for deployment cost.

3 GPTQ and AWQ: post-training quantization

GPTQ (2022)

- Post-training quantization for GPT-style models
- Uses **Hessian-based** second-order optimization
- Quantizes layer-by-layer; compensates errors in remaining weights
- **4-bit GPTQ**: 70B models on consumer GPUs (2× RTX 3090)
- Slightly slower than INT8 due to dequantization overhead

AWQ (2023)

- **Activation-aware Weight Quantization**
- Not all weights matter equally
- Protect top **1% of weights** by activation magnitude
- These "salient" weights drive most of the output signal
- Better perplexity than GPTQ on most benchmarks

Why GPTQ works

Naive rounding: round each weight to nearest integer. GPTQ: after rounding w_i, correct the error in $w_{i+1} \dots w_n$ so the layer output is preserved.

Benchmark: LLaMA-2-7B perplexity

Method	PPL
fp16 baseline	5.47
GPTQ 4-bit	5.85
AWQ 4-bit	5.60

Lower perplexity = better. AWQ is closer to fp16.

3 GGUF: quantized models for CPU inference

The CPU inference problem

- Most developers don't have A100 GPUs
- Can we run LLMs on a MacBook?
- **llama.cpp** (Gerganov, 2023): C/C++ inference for LLaMA-family models
- Highly optimised for CPU SIMD (AVX2, ARM NEON)
- No Python, no CUDA, minimal dependencies

GGUF format

- File format for quantized models (formerly GGML)
- Single self-contained file (weights + metadata + tokenizer)
- Levels: Q2_K, Q3_K, Q4_K, Q5_K, Q8_0
- **Q4_K_M**: 7B model = 4.1 GB. Runs on MacBook Air M1.
- Available for most open models on Hugging Face

Speed on Apple M1 (7B model, Q4)

Device	Tok/s
MacBook Air M1 (CPU)	∼20–30
MacBook Pro M3 Max	∼60–80
RTX 4090 (GPU)	∼120–150
A100 (fp16)	∼80–100

Practical use case

Local dev, privacy-sensitive apps, edge deployment, offline demos. Not for high-throughput production (use vLLM).

3 Quantization trade-off: quality vs. size vs. speed

Precision	Memory (70B)	Relative Speed	Quality Loss
fp32	280 GB	1× baseline	None
fp16	140 GB	1.8×	Negligible
INT8	70 GB	2×	Very small (<1% PPL)
INT4	35 GB	3×	Small (∼3–7% PPL)
INT2	18 GB	4×	Large (often unusable)

The "free lunch" observation

INT8/INT4 are **"almost free"**: small quality loss for 2–4× memory and speed gain. Most production systems run fp16; INT8 is becoming standard.

When quantization fails

- Tasks requiring precise numerical reasoning
- Extreme INT2/INT3 levels
- Models not well-trained to begin with
- Always validate on your task, not just perplexity

5-Minute Break

Your GPU is still running. You, however, need a break.

Step away, grab some water, and let the attention heads rest. See you in five.

4

Batching and Speculative Decoding

4 Static batching vs. continuous batching

Static (naive) batching

- Collect N requests, run them as a batch together
- Simple: one big forward pass for all N
- Problem: sequences have **different lengths**
- If one request needs 2000 tokens and others need 10, the short ones wait for the long one to finish
- GPU idle while waiting for the slowest sequence
- Real-world GPU utilization: **20–30%**

Continuous (iterative) batching

- After each decoding step, check if any sequence is done
- If sequence i finishes, **immediately insert a new request** into that slot
- The batch composition changes *at every step*
- No waiting: as soon as a slot is free, it fills
- GPU utilization: **70–85%**

The result

Continuous batching $\approx$ **3–4**$\times$ **higher throughput** on the same hardware compared to static batching. It is the default in vLLM, TGI (Text Generation Inference), and every serious LLM serving framework.

4 Speculative decoding: the draft model trick

The bottleneck recap

- Large model: slow per-token generation
- We need more tokens per second without quality loss
- Key insight: *verifying* tokens is cheaper than *generating* them

The algorithm

1. **Draft**: small fast model proposes K tokens
2. **Verify**: large model processes all K tokens in **one forward pass** (parallel, not sequential)
3. **Accept/reject**: if large model agrees with draft token i, accept; first disagreement $\Rightarrow$ reject tokens $i \dots K$, resample from large model
4. Repeat from step 1 with accepted tokens as context

Why the verification is parallel

During verification, the large model processes all K draft tokens simultaneously in one pass. This works because the large model computes $P(t_i | t_1 \dots t_{i-1})$ for all i at once using teacher-forcing — it does not need to auto-regressively wait between tokens.

Performance characteristics

- **Speedup**: 2–3$\times$ for greedy decoding
- Best for: code generation, factual recall (predictable outputs)
- Worst for: creative writing (draft model diverges quickly)
- Draft model must be same architecture family

4 Quiz

Q1 — KV Cache Trade-off

You are serving a 7B model on a single A100 (80 GB). The model weights take 14 GB at fp16. Each request uses a 4096-token context. Your KV cache takes approximately 4 GB per request.

How many concurrent requests can you serve before KV cache alone exhausts VRAM? What optimization would you apply first to increase this number?

Q2 — Quantization Choice

Your team has a production summarization service running LLaMA-2-70B at fp16 on 4× A100s. Cost is $40K/month. You are evaluating INT4 quantization.

What is the maximum number of A100s you could reduce to? What quality risk do you need to validate before shipping INT4 to production?

4 Quiz — Answers

A1 — KV Cache Trade-off

Calculation: Available VRAM after weights $= 80 - 14 = 66$ GB. At 4 GB KV cache per request: $\lfloor 66/4 \rfloor = \mathbf{16}$ concurrent requests.

First optimization: PagedAttention (vLLM). Eliminates KV cache fragmentation (up to 90% waste in naive pre-allocation), allowing the same 66 GB to serve significantly more requests.

A2 — Quantization Choice

Max A100 reduction: INT4 halves memory (140 GB $\rightarrow$ 70 GB). Could reduce from 4$\times$ A100 to **1–2$\times$ A100**, saving \$20–30K/month.

Quality risks: (1) Run summarisation benchmarks (ROUGE, BERTScore) on your production set — not just perplexity. (2) Stress-test on your longest documents. (3) A/B test with human evaluators before full rollout.

5

Deployment Choices

5 ONNX and TensorRT: compilation for speed

ONNX (Open Neural Network Exchange)

- Intermediate representation for neural networks
- Export: `torch.onnx.export()`
- Run on any ONNX runtime (CPU, GPU, mobile, edge)
- Good for: cross-platform deployment, model portability
- Not always as fast as native PyTorch on GPU

ONNX Runtime (ORT)

- Microsoft's optimized ONNX inference engine
- Graph optimizations: operator fusion, constant folding
- Backends: CPU, CUDA, DirectML, CoreML, TensorRT
- Often 1.5–2× faster than raw PyTorch on CPU

TensorRT (NVIDIA)

- Optimizing compiler for NVIDIA GPUs
- **Graph optimization**: fuse layers, eliminate redundant ops
- **Precision calibration**: auto INT8 with sample data
- **Kernel auto-tuning**: best CUDA kernel per op shape
- Speedup: **2–4**× over PyTorch on same GPU

When to use each

Use case	Tool
Cross-platform	ONNX + ORT
Max GPU speed	TensorRT
Mobile/edge	TFLite / CoreML
LLM serving	vLLM / TGI

5 Self-hosted vs. API: the decision framework

Dimension	API (OpenAI, Anthropic, Cohere)	Self-Hosted (Llama, Mistral)
Setup	Minutes. No infra.	Days–weeks. GPU cluster.
Cost	Pay-per-token. Expensive at scale.	Fixed GPU cost. Cheap at scale.
Data privacy	Data leaves your infra	Data stays on-prem
Customization	Limited (system prompt, fine-tune API)	Full control (RLHF, LoRA, etc.)
Latency control	You share capacity with all users	Dedicated hardware
Model quality	GPT-4 class (2024 frontier)	LLaMA-3-70B $\approx$ GPT-4 on many tasks
Engineering burden	Near zero	Significant (infra, monitoring, updates)

Use API when

Prototyping, low volume ($<$\$10K/month), no sensitive data, need frontier model quality

Self-host when

High volume ($>$\$10K/month API spend), data sensitivity, need fine-tuning, latency SLA

5 Cost comparison at scale: API vs. self-hosted

Scenario: 100M tokens/day output

GPT-4o API

- Output: \$60/1M tokens
- 100M $\times$ \$60/1M = **\$6,000**/day
- Monthly: **\$180,000**

Self-hosted LLaMA-3-70B on 8$\times$A100

- 8$\times$A100 cloud instance: $\sim$\$25/hour
- Throughput: $\sim$2,000 tokens/sec at continuous batching
- 100M tokens/day = $\sim$14 hours of GPU time
- Cost: 14 $\times$ \$25 = **\$350**/day
- Monthly: $\approx$ **\$10,500**

Savings: 17$\times$ cheaper at this volume

Break-even analysis

- Below $\sim$\$10K/month API: use API (engineering cost too high)
- \$10K–\$50K/month: evaluate case-by-case
- Above \$50K/month API spend: almost always self-host

Hidden self-hosting costs

- 1–2 ML engineers to maintain the stack
- Model updates (new LLaMA versions, safety patches)
- Monitoring, autoscaling, HA infrastructure
- Reserved GPU instances vs. spot (2–3$\times$ cost difference)

5 What We Covered Today

§1 Why LLM Inference Is Hard

- Memory: 175B params = 350 GB at fp16
- Sequential generation: can't parallelize tokens
- Memory-bandwidth bound, not compute-bound
- The cost math: tokens × price × volume

§2 KV Cache and Memory Management

- KV cache: store K, V tensors for all previous tokens
- Memory cost: up to 100 GB per request at scale
- PagedAttention: OS paging for KV cache
- vLLM: 24× throughput improvement

§3 Quantization

- fp32 → fp16 → INT8 → INT4
- GPTQ, AWQ: PTQ techniques
- GGUF for CPU inference (llama.cpp)
- Quality vs. size vs. speed trade-offs

§4 & §5 Batching, Decoding, Deployment

- Continuous batching: 70%+ GPU utilization
- Speculative decoding: 2–3× speedup
- ONNX / TensorRT for compilation
- API vs. self-host decision framework

Thank You!

Serving a 175B model on a MacBook. Context length optional. What could go wrong?

Priyansh Saxena

All materials at:

https://www.priyanshsaxena.com/sst-ml-system-design

